What the model reads, predicts and weighs

How a language model works, and what to do when a session goes wrong

Florian Gärtner

Where we are today

	What this course is for
	Getting it running
	Try it out
	The terminal, and how to read what Claude does
	Lunch
	A working day with Claude Code
→	**What the model reads, predicts and weighs** ← you are here
	A model, a menu, and a loop
	It doesn't remember. It's handed everything again.
	Wrap, done at 17:00

Learning objectives

After this block you will be able to:

1. **Explain** what the model receives when you type, and why your spelling and your language change it.
2. **Explain** why a fluent answer can be invented, and where the source has to come from instead.
3. **Predict** how adding material to a prompt, or moving it, changes what the model uses.
4. **Choose** between rewinding, compacting and starting over when a session goes wrong.

From what you type to the answer

Your text
split into tokens
each one a number

→

The model
predicts the next token
then the next

→

The answer
a likely continuation
not a checked one

→

Everything so far
every word takes weight
a wrong turn too

The answer is a likely continuation of everything that went in.

What arrives

You type this

```
Please check whether the standard errors are clustered.
```

This sentence is split into pieces called tokens, for example like this:

```
Please | check | whether | the | standard | errors | are | clustered | .
```

Strictly, the model receives this:

```
8256    2371    5588    290    7024    10664    553    198429    13
```

Nine tokens. Not nine words. Nine entries in a fixed list.

Measured with tiktoken, OpenAI's published tokeniser for GPT-4o (`o200k_base`). Anthropic does not publish Claude's. Our numbers on these slides come from the script in `exercises/tokeniser`.

Every token is a number

Every model has its own fixed vocabulary of tokens: GPT-4o's has about 200,000, GPT-2's 50,257. Anthropic does not publish the size of Claude's. And Claude has more than one: since Opus 4.7, the same text makes about 30% more tokens.

Your text is cut into the model's vocabulary before anything else happens. Each token is then just its position in the list.

The number itself means nothing. It works like a phone number: it tells you where to look something up, not what you will find there. 198429 is not "more" than 13.

> **The one sentence to keep**
>
> The model never sees your letters, and it never sees your words. It sees a sequence of ID numbers.

Nobody sat down and wrote the list

The vocabulary is learned from a corpus, before the model is trained, by a procedure that is almost embarrassingly simple:

1. Start with every individual character as its own token.
2. Count every adjacent pair in the corpus. Merge the most frequent pair into one new token.
3. Repeat until you have as many tokens as you wanted.

That is the whole algorithm. It is called byte-pair encoding.

First a way to compress files (Gage 1994); since GPT-1, the way OpenAI's models build their vocabulary (Radford et al. 2018). Anthropic's published tokeniser for older Claude models used it too. For current Claude models, Anthropic publishes only token counts.

So the vocabulary is shaped by frequency

Not by grammar. Not by meaning. By how often a sequence of characters happened to occur in the training corpus.

Three things to know:

- **Common words are one token.** Rare words, technical terms and proper nouns are several.
- **The characters next to a word can change its token.** `"the"`, `" the"` and `"(the"` are three different entries.
- **Numbers are cut into pieces, and where depends on the tokeniser.** `2026` is `202` + `6` for GPT-4o, `20` + `26` for GPT-3.

A typo is not a damaged word

```
1    regression      1 token:    regression
2    regresion       2 tokens:   regres  ion
```

Not a lucky pick: of all 116 typos one slip away from “regression”, “coefficient”, “hypothesis” and “significant” (a letter dropped, swapped or doubled), none stayed one token.

You read `regresion` as `regression` without noticing, because you know the word. The tokeniser does not know words, only its vocabulary.

`regresion` is not in the vocabulary, so it is cut into other tokens, which the model met in other contexts during training.

> ⚠ **The mistake this explains**
>
> Thinking the model sees a slightly misspelled word, as you do. The model gets different tokens, and can sometimes put the word back together. Kaplan et al. 2025.

It is not the same price in every language

Split into tokens	Words	Tokens
Do | not | change | the | raw | data | ; | write | the | cleaning | into | a | script | instead | .	13	15
Ä | nd | ere | nichts | an | den | Roh | daten | , | sondern | schre | ibe | die | Bere | inigung | in | ein | Skr | ipt | .	12	20

Over twelve such pairs, German needed 32% more tokens with GPT-4o's tokeniser. On Claude, measured through its own token counts: almost twice as many.

Ours, 2026-10-01 (Claude: Haiku 4.5 +93%, Sonnet and Opus 5.5 +87%). Petrov et al. (NeurIPS 2023), 2,000 sentences, GPT-4's tokeniser: +58% (ours: +63%).

Where this is worth acting on

Long standing instructions, repeated context, anything you send every time. English roughly halves them on Claude. Not for a single question.

How it answers

It predicts the next token

A model has no dictionary. It has never been told what any word means.

It does one thing, over and over: given the tokens so far, it predicts the next one. That token is added, and it predicts again.

The prediction is not one token but a probability for every token in the vocabulary.

A model is made in two steps

1. Pre-training
Vast amounts of text.
It learns to predict the next token, over and over.
Result: a text predictor.

→

2. Post-training
Example conversations, and answers rated by people and by other models (RLHF).
Result: an assistant that answers, refuses, or says it does not know.

Step 2 comes back when we ask why a model guesses.

It always writes something

At every step, one token is picked from those probabilities, and the next step builds on it.

No step checks the result against the world. “I don’t know” comes out only where it is a likely continuation.

Even a perfect predictor would predict what people write, not what is true.

> ⚠ **A common mistake**
>
> A fluent, specific answer is not evidence that it is right. Fluency is what the model is built to produce.

A reference is a likely continuation too

Ask for a source, and the model continues the way sources look: author names, a year, a journal, a plausible title.

Short literature reviews, 636 references	GPT-3.5	GPT-4
Fabricated: the work does not exist	55%	18%

Walters & Wilder 2023.

Newer models do better. The step that writes a reference still checks nothing.

Why it guesses

Training, and the tests models are scored on, reward a confident answer over “I don’t know”. Like a multiple-choice exam that gives no points for a blank.

Kalai et al. 2025.

The habit

Never let the model be the source. Let it fetch the paper, run the script or search the database, and keep what it found.

Your kit asks Claude for the same: copy each reference from a source it opened, and mark what it could not check “[unverified]”.

Check the references Claude gives you

Your project · free time

- Ask Claude for three papers on your project's topic, and tell Claude not to search the web.
- Then ask Claude to look each one up. Which exist, and are they cited correctly?

What this means for your prompt

Every word takes weight from the others

Not every word in a prompt matters equally for what comes next. To predict the next token, the model weighs every token so far: how relevant is each one here? This weighing is called **attention**.

The weights are positive and add up to 1. A word that does not matter still gets a small weight.

> **The idea to remember**
>
> Every word you put in a prompt takes weight away from every other word. Irrelevant material is not skipped. It is included at a low weight, and it bids against the material you wanted.

Length alone hurts too, even with nothing irrelevant in the prompt.

A long session works worse, on average

Context rot. Studies give a model the same task with more and more text in front of it, and its answers get worse. Anthropic's documentation says the same of Claude.

Levy, Jacoby & Goldberg 2024; Du et al. 2025; Modarressi et al. 2025; Anthropic, Claude docs, 2026.

Lost in the middle. Models use what stands at the start and at the end of a long text better than what stands in the middle.

Liu et al. 2024.

In a long session, what you asked for an hour ago is in the middle.

Instructions first, or last?

Anthropic's advice for Claude: long documents first, the question and the instructions after them.
Up to 30% better in its tests (Claude prompting docs).

Claude Code itself puts its standing instructions first: the system prompt and `CLAUDE.md` sit at the start of every session. When Claude reads a file or loads a skill for you, that text lands after your question.

Both may be right for the same reason. Models use the start and the end of a long text best, and the middle least (lost in the middle, the slide before). `CLAUDE.md` sits at the start, and your question near the end. Liu et al. 2024.

> **The habit**
>
> Paste a long text first and ask your question after it. Rules that must always hold go in `CLAUDE.md`, at the start.

When it goes wrong

Hard to notice in your own session

In your own session you will rarely notice the slow decline, because you never get the same question answered in a fresh session to compare.

> **Rule of thumb**
>
> Manage a session by rule, not by feel.

How full is it?

`/context` shows what fills the session right now, and how much room is left.

`/usage` shows what the session has used against your plan's limits.

Your status line shows how full it is all the time, on its second line. With your kit, Claude watches it too. Past half full, Claude suggests `/session-end` and a fresh session.

> **Rule of thumb**
>
> Around half full, start thinking about a clean break. Do not wait for the limit.

A wrong turn stays in

Say Claude misread your data early: it thinks `income` is in euros when it is in thousands.

You correct it. Claude agrees. Ten turns later the old reading is back.

Models that take a wrong turn early in a conversation tend not to recover.

Laban, Hayashi, Zhou & Neville 2025.

The correction and the mistake are both in what Claude is handed every turn, and the mistake came first.

What would work better than correcting it?

Rewind past it

`Esc Esc`, or `/rewind`, opens a list of your earlier prompts. Pick the one before the mistake.

You can take the conversation back, the file edits back, or both. Then ask again, better.

> ⚠ **Rewind undoes Claude's own file edits, not the commands it ran**
>
> A file a command deleted stays deleted. Edits a subagent made usually stay too. Nothing that left your machine comes back.

Tell Claude what you did

Claude Code tells Claude about some of what you do, such as switching plan mode on or off. Not about everything.

If you rewind the conversation but keep the file edits, your files contain changes Claude no longer remembers.

If it matters, **say it**: *"I rewound, but kept your edits to* `clean.R`*."*

Compact, or start over

	What it does	When
`/compact`	Swaps the session for a summary	going well but out of room, between steps
`/clear`	Empties the session	going badly, or a new topic

You can tell `/compact` what to keep: `/compact keep the variable definitions`.

Before either: `/session-end`, so what you need next time is saved.

> **Rule of thumb**
>
> If you have corrected the same point twice, start over. With the kit, Claude suggests it itself.

How full is your session?

Your project · free time

- Type `/context` in your project's session. How much room is left?
- Press `Esc Esc` and look at the list of your earlier prompts. Pick one, then choose **Never mind** to leave without changing anything.

Discussion

1. A colleague says Claude Code's references are safe now, because it searches the web. Do you agree?
2. Your question is about one of four papers. Do you give Claude all four, or only that one? When are the other three worth it?
3. Your `CLAUDE.md` says to always use `fixest`. Two hours into a session, Claude uses `lm()`. What do you do?

What to take away

1. The model reads tokens, not words. Spell what matters, and English costs less.
2. The model writes the likely next token. A reference or a number can be invented, so the source has to come from outside.
3. Every word in a prompt takes weight from every other. Irrelevant material is not skipped.
4. Paste a long text first and ask your question after it.
5. Manage a session by rule, not by feel. Rewind past a mistake instead of correcting on top of it.
6. Rewind undoes Claude’s own file edits, not the commands it ran or a subagent’s edits.